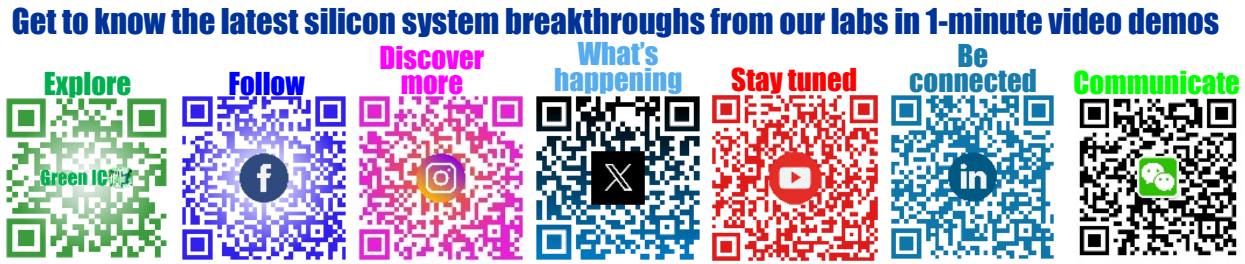
EE2026
Digital Design

BOOLEAN FUNCTION MINIMIZATION

Massimo ALIOTO
Dept of Electrical and Computer Engineering
Email: massimo.alioto@nus.edu.sg



Outline

- Gate-level design and Boolean function minimization
- Karnaugh maps (K-maps)
- Boolean function simplification using K-maps
- Considerations on gate-level implementation

Gate-Level Design with Minimum Complexity

- Minimize Boolean function before gate-level design is crucial
  - Lower cost (fit smaller FPGA), faster (fewer gate delays), lower power (fewer gates)
  - Reduce Boolean function to its minimal form

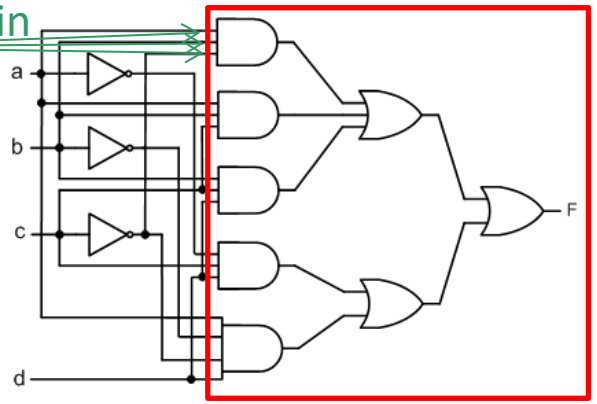
- Definition of simplified Boolean Function (recap)
  - 1) Minimal number of **terms**, 2) minimal number of **literals** in each term

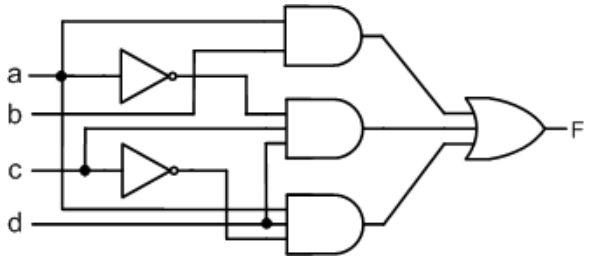
literals (variables, complemented or not)

$$F(a,b,c,d) = ab\bar{c} + abc + bcd + \bar{a}cd + a\bar{b}\bar{c}d$$

terms

$$= ab + \bar{a}cd + a\bar{c}d$$



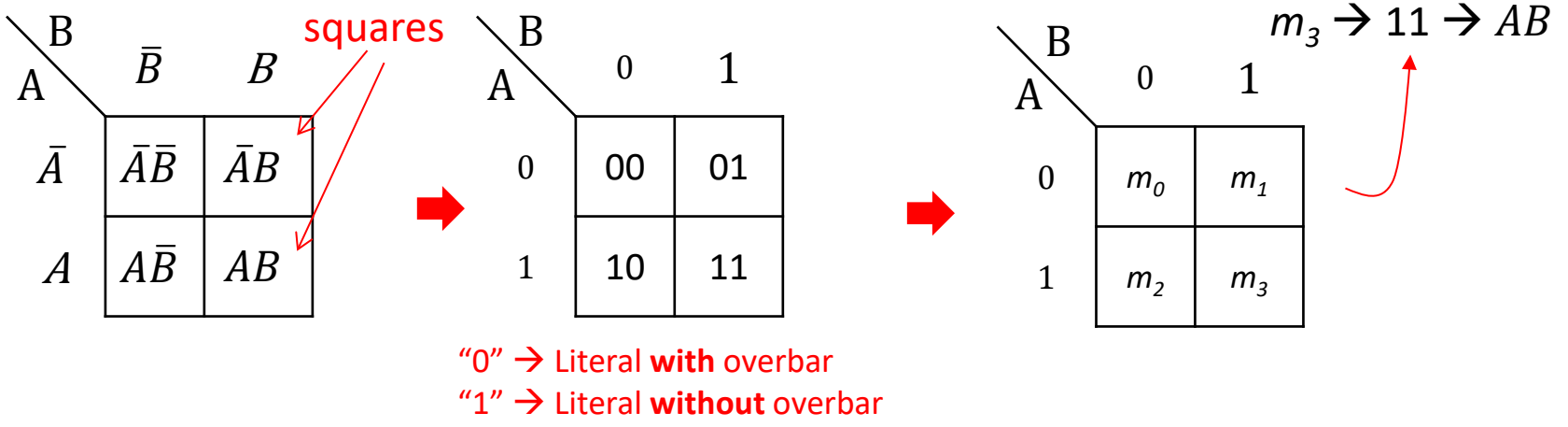


- Impact on gate-level implementation
  - 1) Minimal gat`e count,2) minimal fan-in in each logic gate (gate complexity)

Karnaugh Maps (K-Maps)

- K-map is a diagram that consists of a number of squares, each representing one SOP minterm (or POS maxterm) of a Boolean function
  - The SOP form (POS) can be expressed as a sum of minterms (maxterms) in the map
  - n-variable Boolean function has maximum 2^n minterms (maxterms)

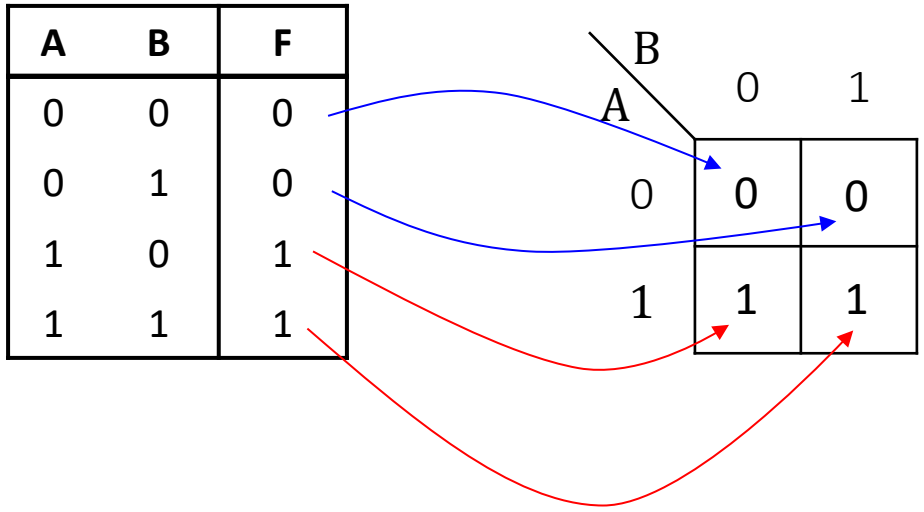
Two-variable K-map:
(maximum 4 minterms)



"0" → Literal **with** overbar
"1" → Literal **without** overbar

Convert Truth table → K-map

- K-map is a two-dimensional representation of the truth table
  - Each row of in truth table corresponds to one square in the k-map
  - If the term in a row is a minterm of the function (F=1), place a “1” in the corresponding square of the K-map, otherwise (maxterm), place a “0”
  - Equivalent to truth table (just rearranged)



Three- and Four-Variable K-Maps

- In K-maps, any two adjacent squares differ by only one literal (“logically adjacent”), same in first-last row and column

Three-variable K-map

A \ BC	$\bar{B}\bar{C}$	$\bar{B}C$	BC	$B\bar{C}$
$\bar{A}$	$\bar{A}\bar{B}\bar{C}$	$\bar{A}\bar{B}C$	$\bar{A}BC$	$\bar{A}B\bar{C}$
A	$A\bar{B}\bar{C}$	$A\bar{B}C$	ABC	$AB\bar{C}$

A \ BC	00	01	11	10
0	000	001	011	010
1	100	101	111	110

A \ BC	00	01	11	10
0	m_0	m_1	m_3	m_2
1	m_4	m_5	m_7	m_6

Four-variable K-map

AB \ CD	00	01	11	10
00	0000	0001	0011	0010
01	0100	0101	0111	0110
11	1100	1101	1111	1110
10	1000	1001	1011	1010

AB \ CD	00	01	11	10
00	m_0	m_1	m_3	m_2
01	m_4	m_5	m_7	m_6
11	m_{12}	m_{13}	m_{15}	m_{14}
10	m_8	m_9	m_{11}	m_{10}

Boolean Functions (Canonical) ↔ K-map

- Practice: represent function in canonical form through K-map
  - SOP: just place a “1” in squares representing its minterms

$$F = \overline{A}B + AB + A\overline{B}$$

A \ B	0	1
0	0	1
1	1	1

Write the Boolean expression from the K-map

A \ B	0	1
0	0	(1)
1	(1)	0

$\boldsymbol{F} = ?$

in SOP: write F as sum of the minterms (squares with “1”)

$$\boldsymbol{F} = \overline{A}B + A\overline{B}$$

Boolean Functions (Canonical) ↔ K-map

Represent the following function on K-map:

$$F = \bar{A}BC + AB\bar{C} + \bar{A}\bar{B}C + \bar{A}\bar{B}\bar{C}$$

A \ BC	00	01	11	10
0	1	1	1	0
1	0	0	0	1

$$F = \bar{A}B\bar{C}D + \bar{A}\bar{B}CD + AB\bar{C}\bar{D} + \bar{A}\bar{B}C\bar{D} + A\bar{B}C\bar{D} + A\bar{B}CD + A\bar{B}\bar{C}D + \bar{A}\bar{B}\bar{C}\bar{D}$$

AB \ CD	00	01	11	10
00	1	0	1	1
01	0	1	0	0
11	1	0	0	0
10	0	1	1	1

Write the Boolean expression for the function in K-map:

A \ BC	00	01	11	10
0	1	0	0	0
1	0	1	0	0

$\boldsymbol{F} = ?$ $\quad \boldsymbol{F} = \overline{A} \cdot \overline{B} \cdot \overline{C} + A \cdot \bar{B} \cdot C$

AB \ CD	00	01	11	10
00	0	1	0	0
01	0	0	0	1
11	0	1	0	0
10	0	0	0	0

$\boldsymbol{F} = ?$ $\quad \boldsymbol{F} = \overline{A\bar{B}\bar{C}D} + \overline{A}BC\overline{D} + AB\overline{C}D$

Boolean Functions ↔ K-map

- Practice: represent function in non-canonical form through K-map
  - No need to expand products to minterms (sums to maxterms)
  - Just identify (groups of) squares corresponding to each term

$$F = \overline{A}B + AB\overline{C} + \bar{A}\bar{B}C$$

$$\overline{A}B = \overline{A}B(C + \overline{C}) = \overline{A}BC + \overline{A}B\overline{C}$$

A \ BC	00	01	11	10
0	0	1	1	1
1	0	0	0	1

Or $\overline{A}B$ → 01, $C = 0\ or\ 1$

or just fill the truth table and derive the K-map

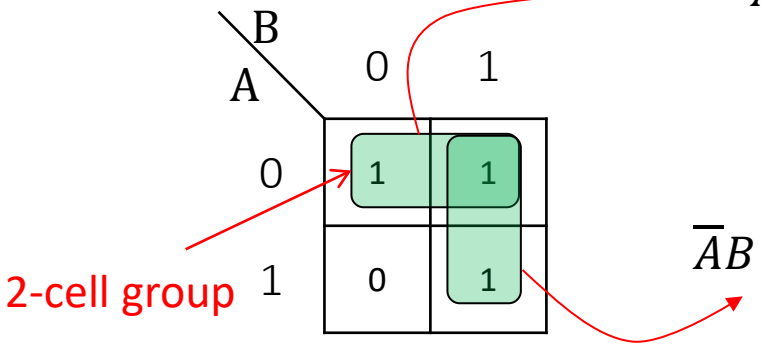
$$F = \bar{A} + \bar{A}\bar{B}CD + B\bar{C}\bar{D}$$

AB \ CD	00	01	11	10
00	0	0	1	0
01	1	0	0	0
11	1	1	1	1
10	1	1	1	1

Boolean Function Simplification Using K-Maps

- Observation: 2 adjacent squares differ by one literal → simplified into 1

Simplify: $F = \bar{A}B + AB + \bar{A}\bar{B}$



$\bar{A}B + \bar{A}\bar{B} = \bar{A}(B + \bar{B}) = \bar{A}$

literals by which they differ: eliminated

common literal: remains

$\bar{A}B + AB = B(\bar{A} + A) = B$

literals by which they differ: eliminated

common literal: remains

MSOP:

$F = \bar{A} + B$

- Variable that is common remains
- Variable that changes is eliminated
- Check using Boolean manipulations

$$F = \bar{A}B + AB + \bar{A}\bar{B}$$
$$= \bar{A} + AB$$
$$= \bar{A} + B$$

Boolean Function Simplification Using K-Maps

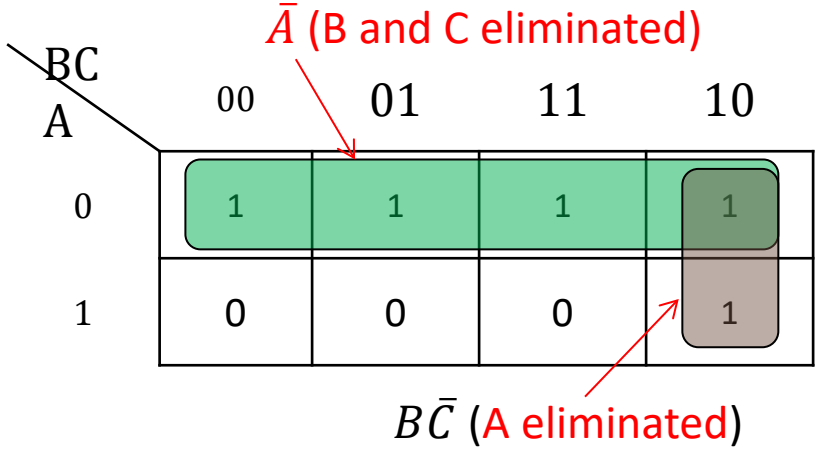
Three-variables:

$$F = \bar{A}\bar{B}\bar{C} + \bar{A}\bar{B}C + \bar{A}BC + \bar{A}B\bar{C} + AB\bar{C}$$

$$F = \bar{A} + B\bar{C}$$

$\bar{A}\bar{B}\bar{C} + \bar{A}\bar{B}C + \bar{A}BC + \bar{A}B\bar{C} \rightarrow \bar{A}\bar{B}(\bar{C} + C) + \bar{A}B(C + \bar{C})$
$\rightarrow \bar{A}\bar{B} + \bar{A}B \rightarrow \bar{A}(\bar{B} + B) \rightarrow \bar{A}$

$\bar{A}B\bar{C} + AB\bar{C} \rightarrow (\bar{A} + A)B\bar{C} \rightarrow B\bar{C}$

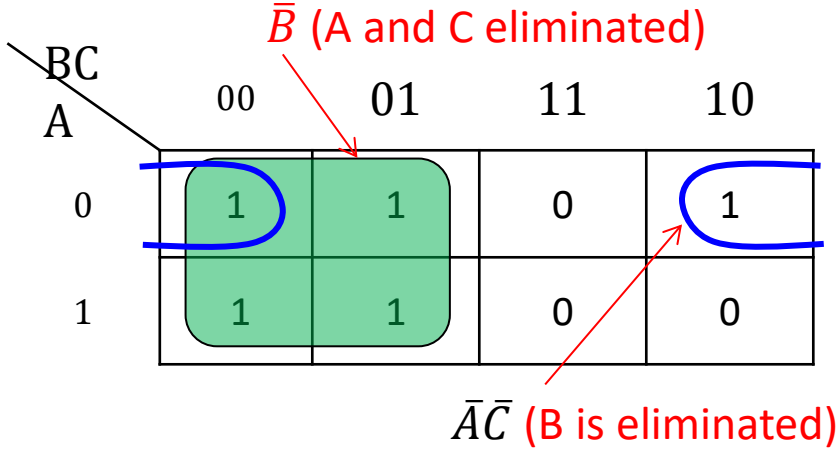


$$F = \bar{A}\bar{B}C + A\bar{B}\bar{C} + A\bar{B}C + \bar{A}\bar{B}\bar{C} + \bar{A}B\bar{C}$$

$$F = \bar{B} + \bar{A}\bar{C}$$

$\bar{A}\bar{B}C + A\bar{B}\bar{C} + A\bar{B}C + \bar{A}\bar{B}\bar{C} \rightarrow \bar{A}\bar{B}(C + \bar{C}) + A\bar{B}(\bar{C} + C)$
$(\bar{A} + A)\bar{B} \rightarrow \bar{B}$

$\bar{A}\bar{B}\bar{C} + \bar{A}B\bar{C} \rightarrow \bar{A}(\bar{B} + B)\bar{C} \rightarrow \bar{A}\bar{C}$



- Group adjacent cells where only one variable changes, so that it can be eliminated

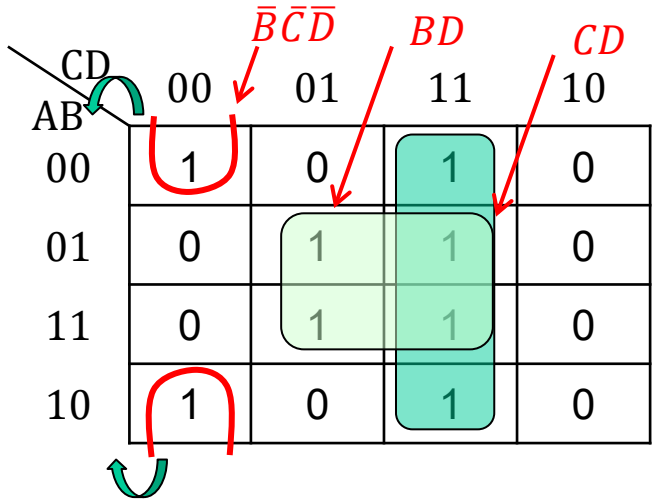
Boolean Function Simplification Using K-Maps

Four-variables:

$$F = \bar{A}\bar{B}\bar{C}\bar{D} + \bar{A}\bar{B}CD + \bar{A}B\bar{C}D + \bar{A}BCD + AB\bar{C}D + ABCD + A\bar{B}\bar{C}\bar{D} + A\bar{B}CD$$

$$\downarrow$$

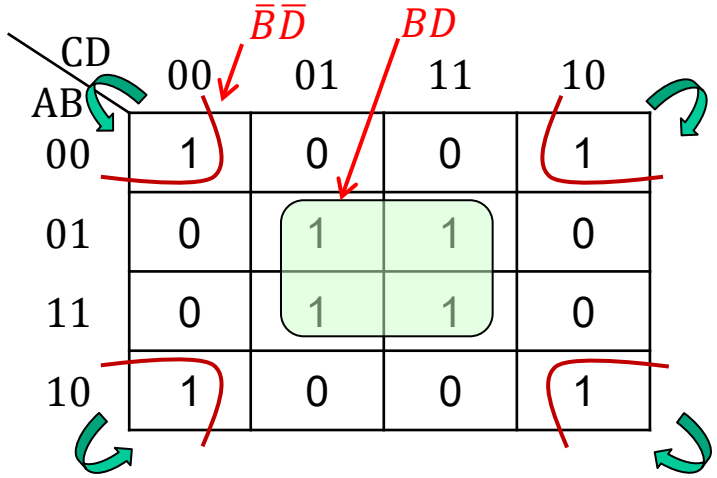
$$F = \bar{B}\bar{C}\bar{D} + BD + CD$$



$$F = \bar{A}\bar{B}\bar{C}\bar{D} + \bar{A}\bar{B}C\bar{D} + \bar{A}B\bar{C}D + \bar{A}BCD + AB\bar{C}D + ABCD + A\bar{B}\bar{C}\bar{D} + A\bar{B}C\bar{D}$$

$$\downarrow$$

$$F = \bar{B}\bar{D} + BD$$



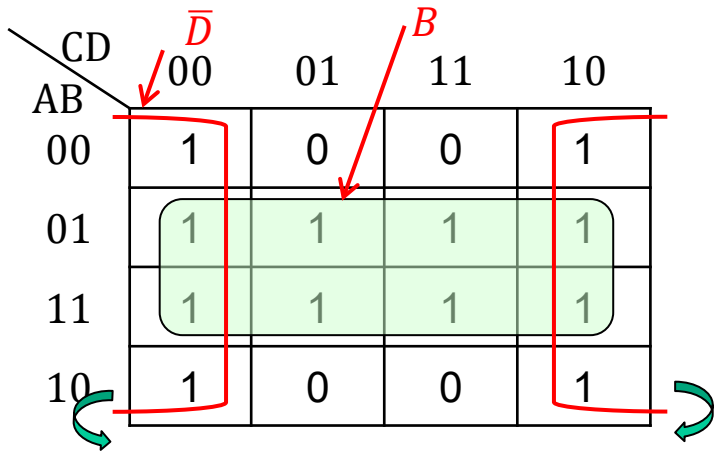
Boolean Function Simplification Using K-Maps

Four-variables:

$$F = \bar{A}\bar{B}\bar{C}\bar{D} + \bar{A}\bar{B}C\bar{D} + \bar{A}B\bar{C}\bar{D} + \bar{A}B\bar{C}D$$
$$+\bar{A}BCD + \bar{A}BC\bar{D} + AB\bar{C}\bar{D} + AB\bar{C}D$$
$$+ABCD + ABC\bar{D} + A\bar{B}\bar{C}\bar{D} + A\bar{B}C\bar{D}$$

$$\downarrow$$

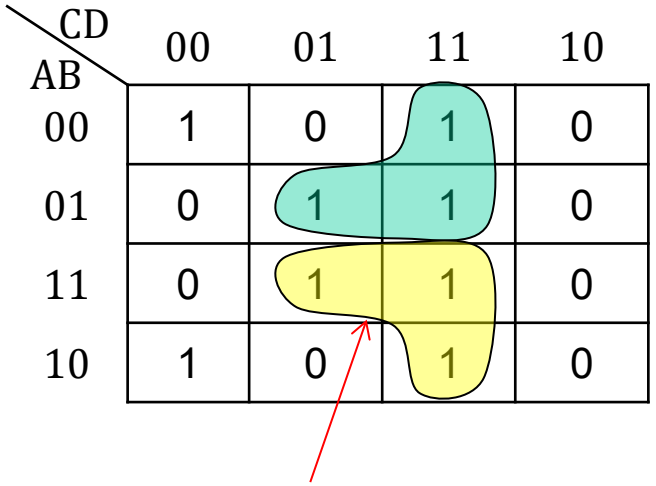
$$F = B + \overline{D}$$



Grouping rules:

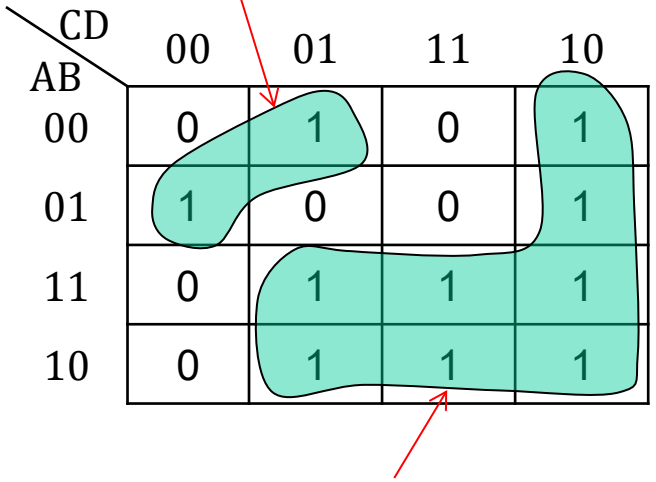
- Group the squares that only contains “1”
- Groups must be either horizontal or vertical (diagonal is invalid)
- Group size is always 2^n, that is, 2, 4, 8, …
- Group should be as large as possible (contains as many as squares with “1” as possible)
- Each square with “1” must be part of a group if possible
- Simplified term retains those variables that don’t change value
- Variables that change value in the group are eliminated

Invalid Groupings



Squares in the group are not in power of two

two variable change value



not horizontal or vertical

Don’t-Care Conditions

- So far, all combination of input variables assumed to be valid
  - n-variable Boolean function $\rightarrow 2^n$ input combinations to be considered

- In practical cases, some variable combinations never appear
  - Example: BCD code (0…9 valid, 10…15 invalid)
  - Example: not all processor words point at existing instructions

Binary-Coded Decimal (BCD)

Decimal Symbol	BCD Digit
0	0000
1	0001
2	0010
3	0011
4	0100
5	0101
6	0110
7	0111
8	1000
9	1001

- Invalid input values are called *don’t-care conditions*
  - Marked with “X” or “-” in K-map
  - For minimization, X can take either “1” or “0”
    - Can choose freely, based on pure convenience for simplification purposes
    - Each d.c.c. is set independently from all others

Minimization with Don't-Care Conditions: SOP

- Choose value that allows to expand groups of squares as much as possible
  - Do not do it otherwise (it adds further terms, more complex)

$$F = \bar{A}\bar{B}\bar{C}\bar{D} + \bar{A}\bar{B}C\bar{D} + \bar{A}B\bar{C}D + \bar{A}BC\bar{D} + AB\bar{C}D + ABC\bar{D} + A\bar{B}\bar{C}\bar{D} + A\bar{B}C\bar{D}$$

$$\downarrow$$

$$F = B + \overline{D}$$

*Treat X = 1 and group the squares as usual

CD / AB	00	01	11	10
00	1	0	0	1
01	X	1	X	1
11	X	1	X	1
10	1	0	0	1

Assume X = 1

Minimization with K-Maps: POS

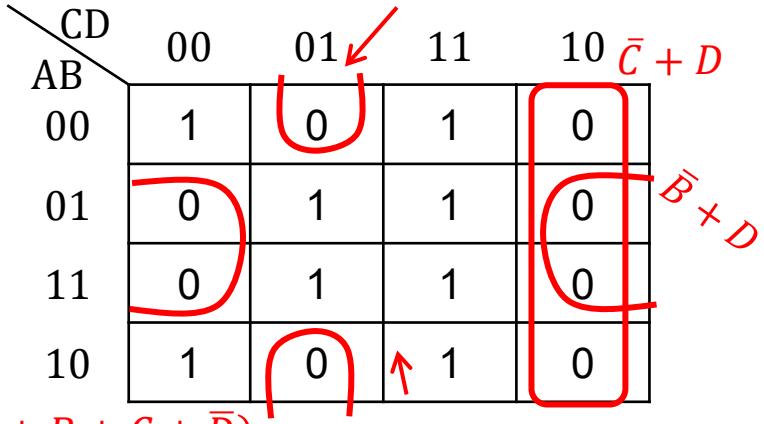
- Dual procedure compared to SOP (swap 0-1, swap products with sums)
  - Group the squares that only contains “0”
  - Form an OR term (sum) for each group, instead of a product
  - Value “1”, instead of “0”, represent complement of the variable
  - Follow similar grouping rules that we discovered in SOP

$$F = (A + B + C + \bar{D})(A + B + \bar{C} + D)$$
$$(A + \bar{B} + C + D)(A + \bar{B} + \bar{C} + D)$$
$$(\bar{A} + \bar{B} + C + D)(\bar{A} + \bar{B} + \bar{C} + D)$$
$$(\bar{A} + B + C + \bar{D})(\bar{A} + B + \bar{C} + D)$$

$$\downarrow$$

$$F = (B + C + \bar{D}) \cdot (\bar{C} + D) \cdot (\bar{B} + D)$$

maxterm-input correspondence: complement literals if 1



$$(A + B + C + \bar{D}) \cdot (\bar{A} + B + C + \bar{D})$$
$$= A\bar{A} + A \cdot (B + C + \bar{D}) + (B + C + \bar{D}) \cdot \bar{A} + (B + C + \bar{D}) \cdot (B + C + \bar{D}) = (B + C + \bar{D})$$

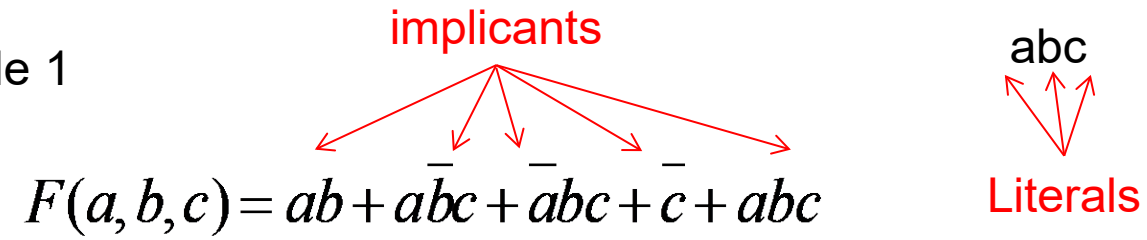
  - No way to know if SOP is simpler than POS upfront (or vice versa)

→ Need to implement both and compare terms and literals

A Systematic Procedure to Minimize with K-Maps

- Some terminology
  - Implicant, prime implicant and essential implicant
- Implicant of a Boolean function
  - Each product term in SOP is called an implicant of the function

Example 1



Example 2

A \ BC	00	01	11	10
0	1	1	0	0
1	1	1	1	0

How many implicants?

A Systematic Procedure to Minimize with K-Maps

- Prime implicant
  - Implicant that cannot be combined with another term to eliminate a variable
  - Graphically: it cannot be enclosed within a larger square/rectangle in K-map

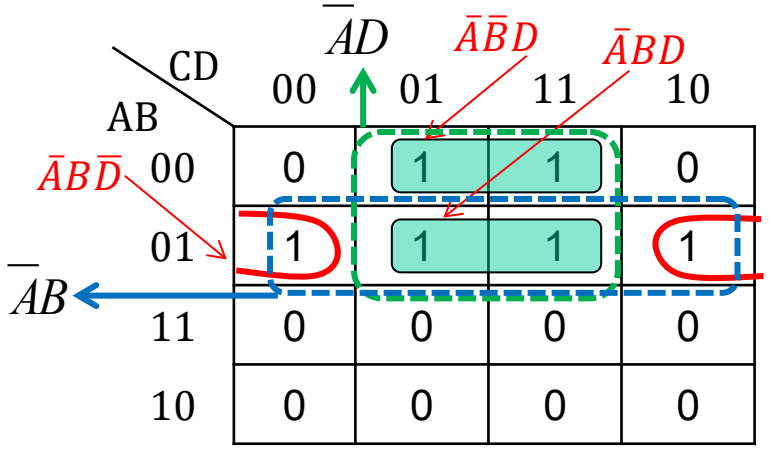
Example 1

non-prime implicant (already contained in AB or BC)

$$F = AB + ABC + BC$$

prime implicants

Example 2



$\bar{A}\bar{B}D$, $\bar{A}BD$ and $\bar{A}B\bar{D}$

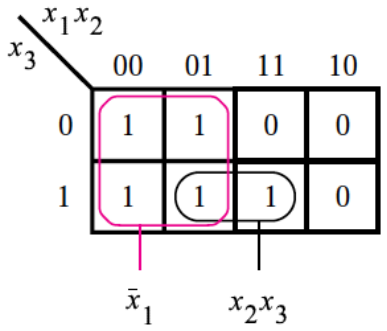
are implicants, but not prime implicants (can be grouped into larger groups of 4)

$\bar{A}D$ and $\bar{A}B$ are essential prime implicants

graphically: prime implicant grouping cannot be expanded further (but could overlap with other prime implicants)

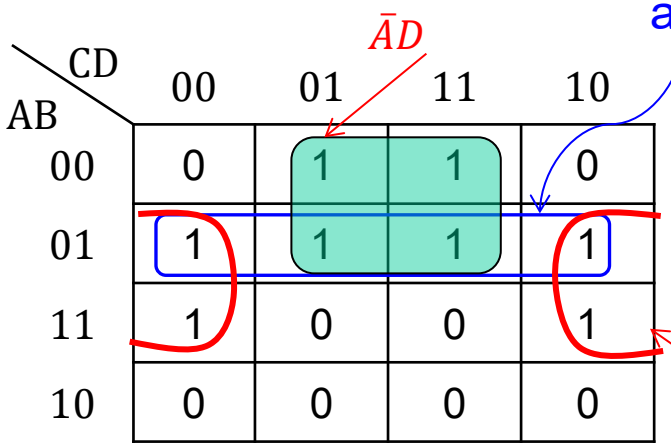
A Systematic Procedure to Minimize with K-Maps

- Essential prime implicant
  - Prime implicant that is not included in any other prime implicant



Both $\bar{x}_1$ and x_2x_3 are essential prime implicants

Prime implicant (not an essential prime implicant, already covered by the other two implicants)



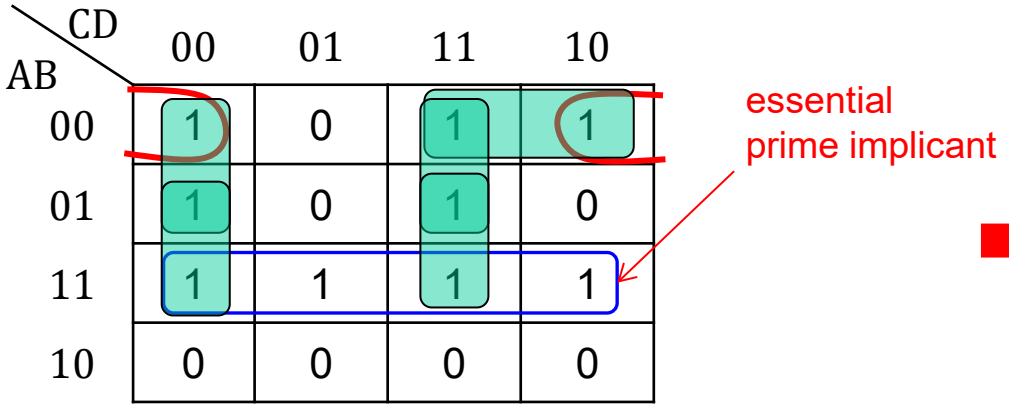
Essential prime implicants: $\bar{A}D$ and $B\bar{D}$

graphically: essential prime implicant is needed to cover some 1 (i.e., it does not completely overlap with other implicants)

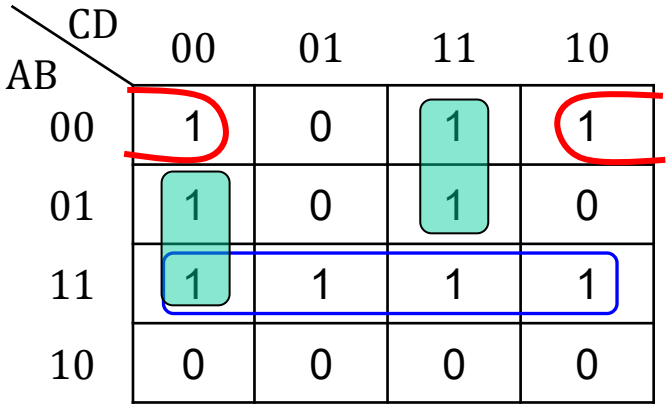
A Systematic Procedure to Minimize with K-Maps

- Resulting systematic procedure for K-map minimization in SOP form
  - Finding all prime implicants of the function
  - Select essential prime implicants (expand groups as much as possible)
  - Find a minimal subset of these prime implicants that covers all of the minterms of the function

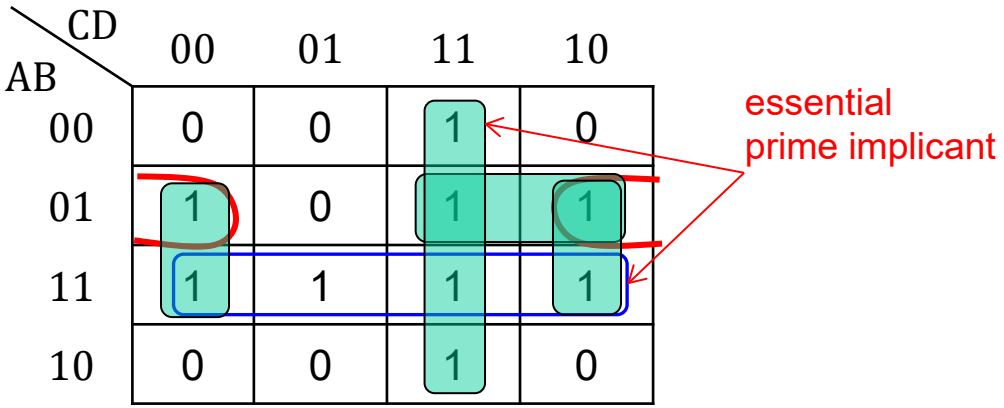
select the essential prime implicant with minimum set of prime implicants



all implicants including **one** essential prime implicant

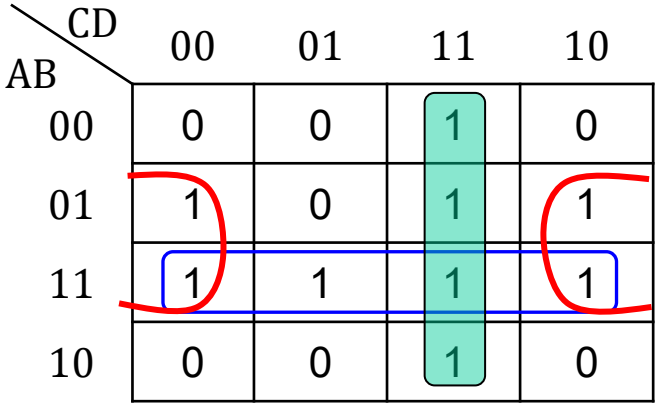


A Systematic Procedure to Minimize with K-Maps

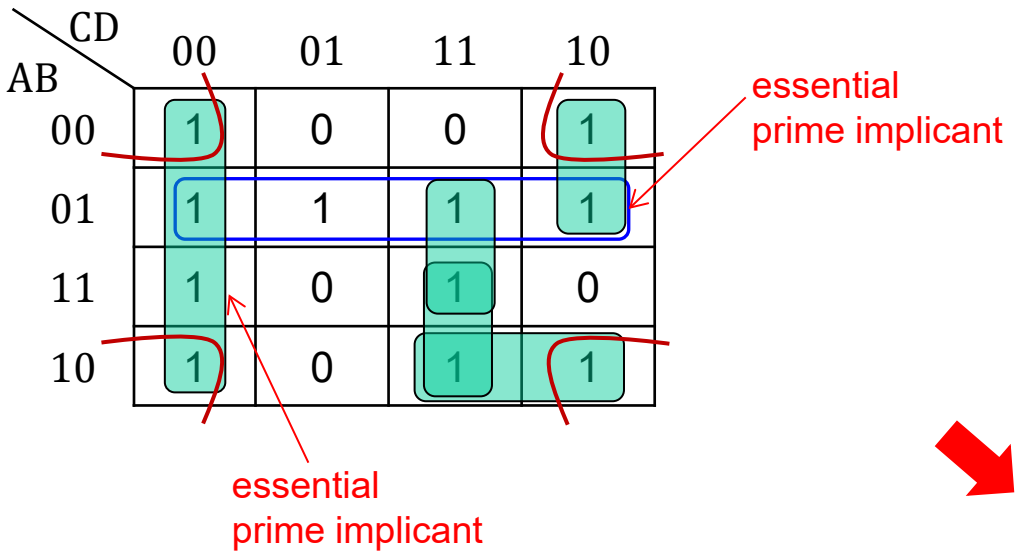


All implicants including
two essential prime implicant

select the essential prime implicant
with minimum set of prime implicants

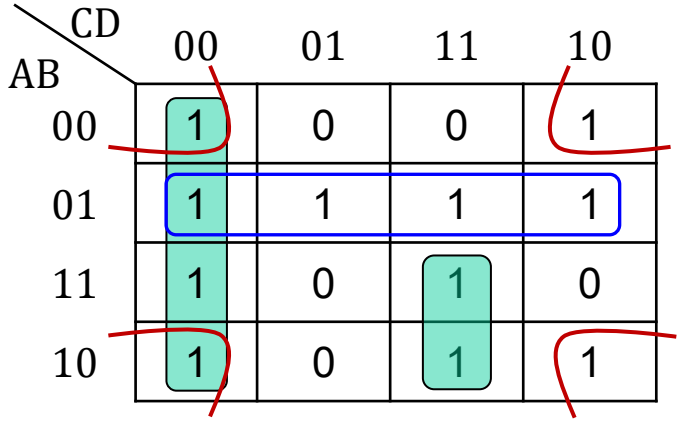


A Systematic Procedure to Minimize with K-Maps



all implicants including
two essential prime implicant

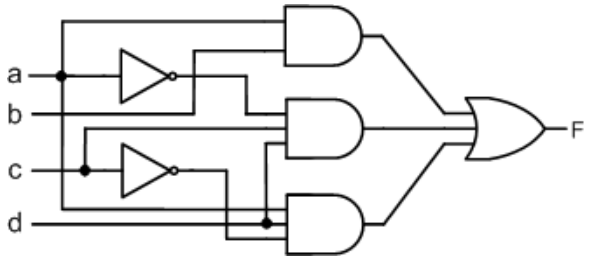
select the essential prime implicant
with minimum set of prime implicants



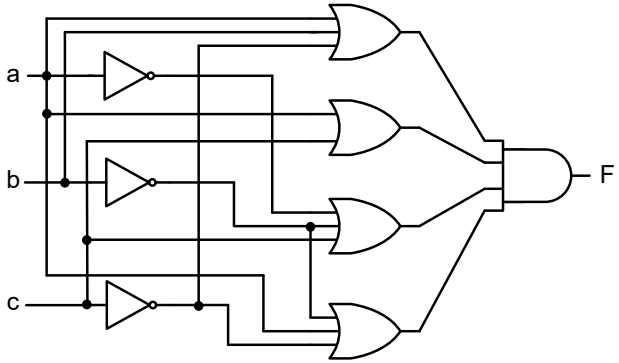
Gate-Level Implementation

- Resulting minimal form of Boolean function → gate-level implementation
  - Straightforward (see previous lectures)

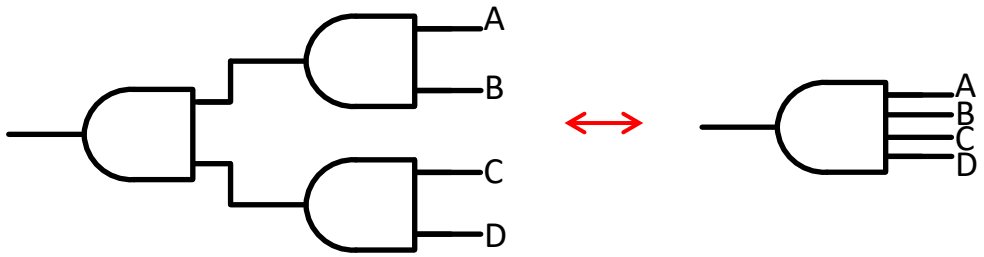
SOP



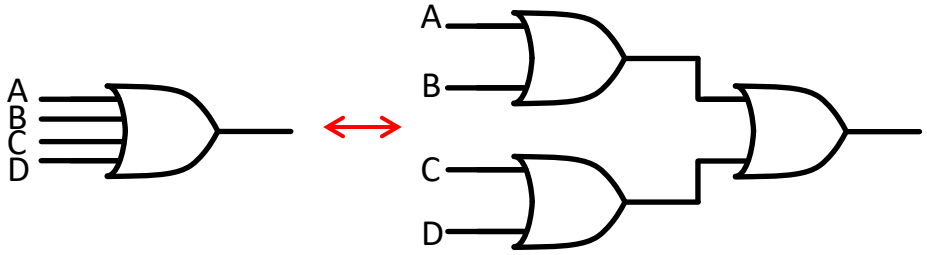
POS



- Extra constraints
  - Limited fan-in: use associative law



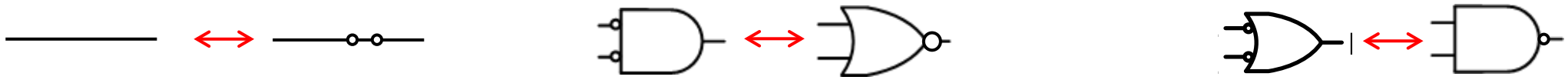
$$A \cdot B \cdot C \cdot D = (A \cdot B) \cdot (C \cdot D)$$



$$A + B + C + D = (A + B) + (C + D)$$

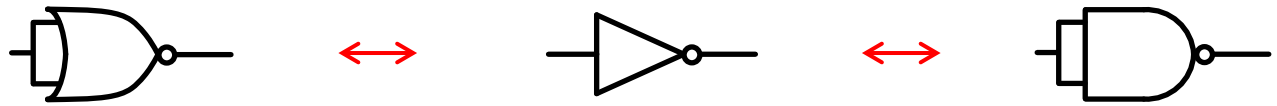
Gate-Level Implementation

- Limited availability of logic gates (e.g., NAND vs. NOR), or high performance / power / area penalty → **bubble pushing**



- Unavailable inverter gates: replace by NAND or NOR
  - short circuit all inputs (any fan-in)

A	B	$\overline{A+B}$
0	0	1
1	1	0

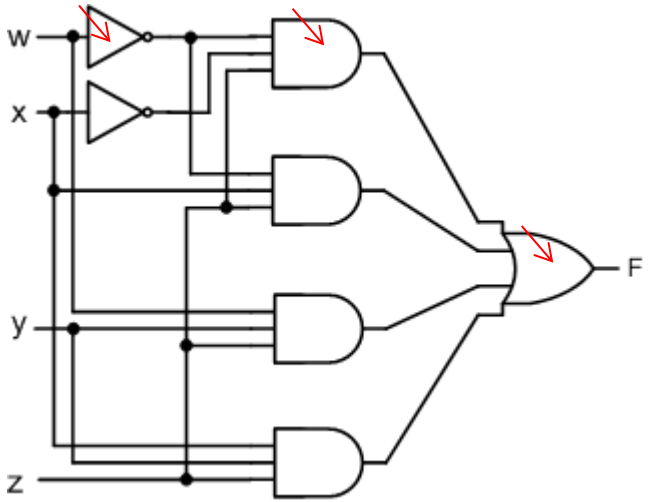


A	B	$\overline{A \cdot B}$
0	0	1
1	1	0

- Negative logic, active-low signals: just complement
  - opposite representation as positive logic

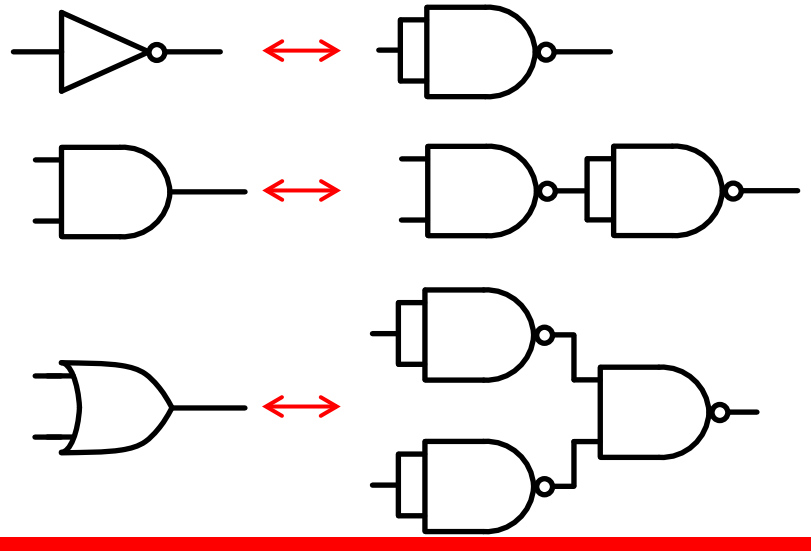
NAND and NOR as Universal Logic Gates

- NAND (and NOR) gates are functionally complete
  - Any Boolean function can be implemented with NAND-only and NOR-only
  - Just consider SOP (dual for POS) form of any Boolean function



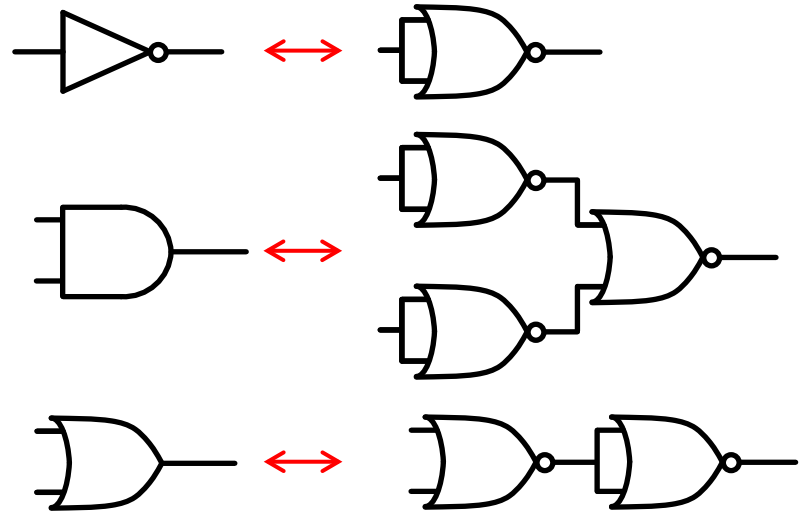
NAND-only

replace NOT, AND and OR with NAND



NOR-only

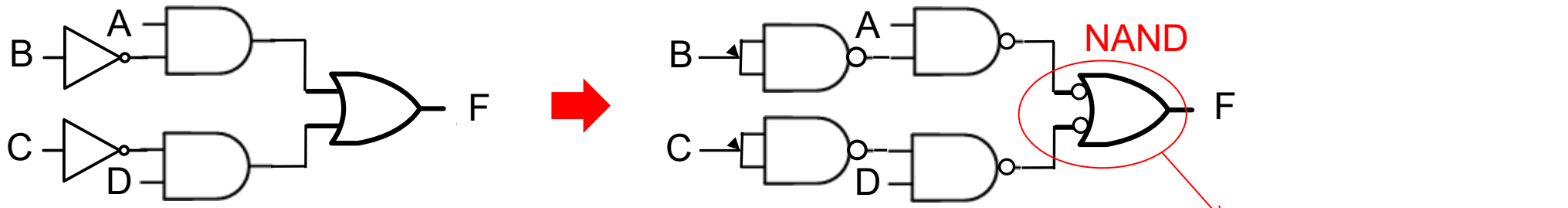
replace NOT, AND and OR with NAND



NAND-Only Gate-Level Design

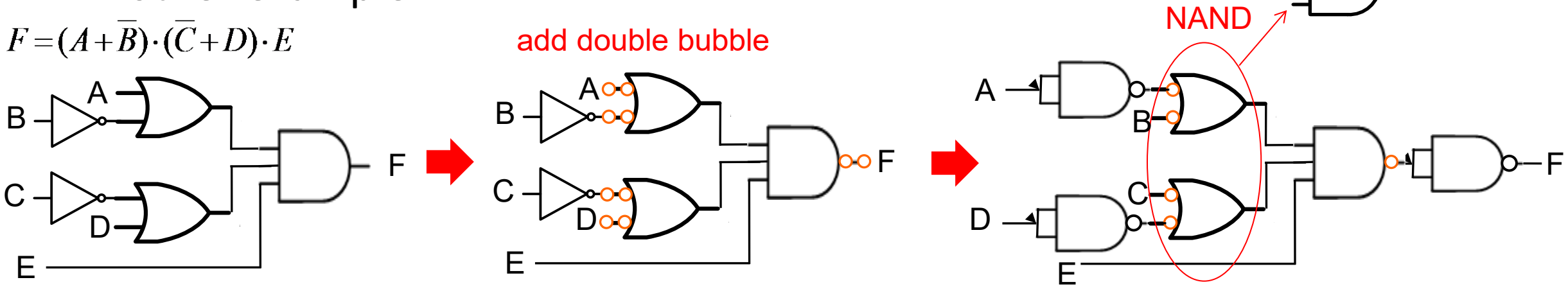
- Example
  - Replace the OR gate with NAND gate (bubble pushing) and balance the bubble(s)
  - Replace the inverter with NAND gate

$$F = A\bar{B} + \bar{C}D$$



- Another example

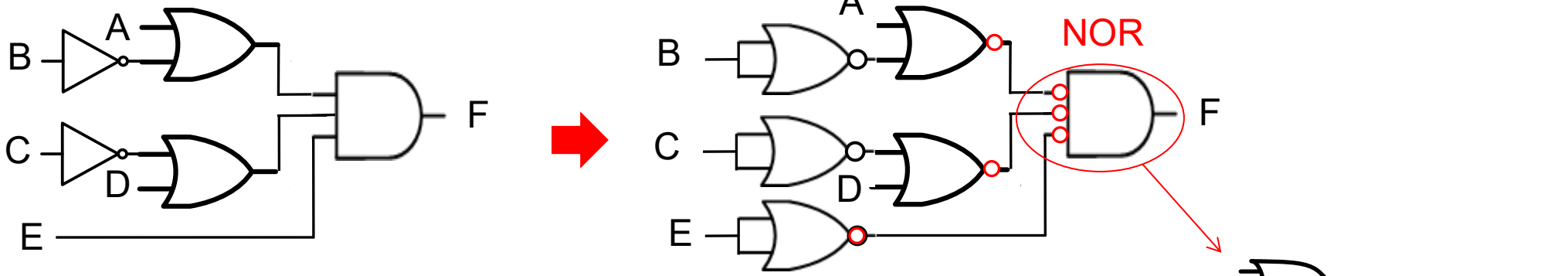
$$F = (A + \bar{B}) \cdot (\bar{C} + D) \cdot E$$



NOR-Only Gate-Level Design

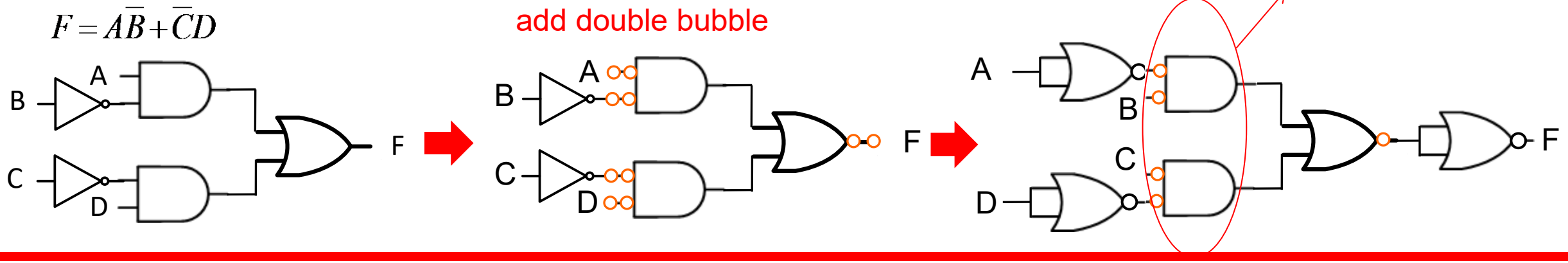
- Example
  - Replace the AND gate with NOR gate (bubble pushing) and balance the bubble(s)
  - Replace the inverter with NOR gate

$$F = (A + \overline{B}) \cdot (\overline{C} + D) \cdot E$$



- Another example

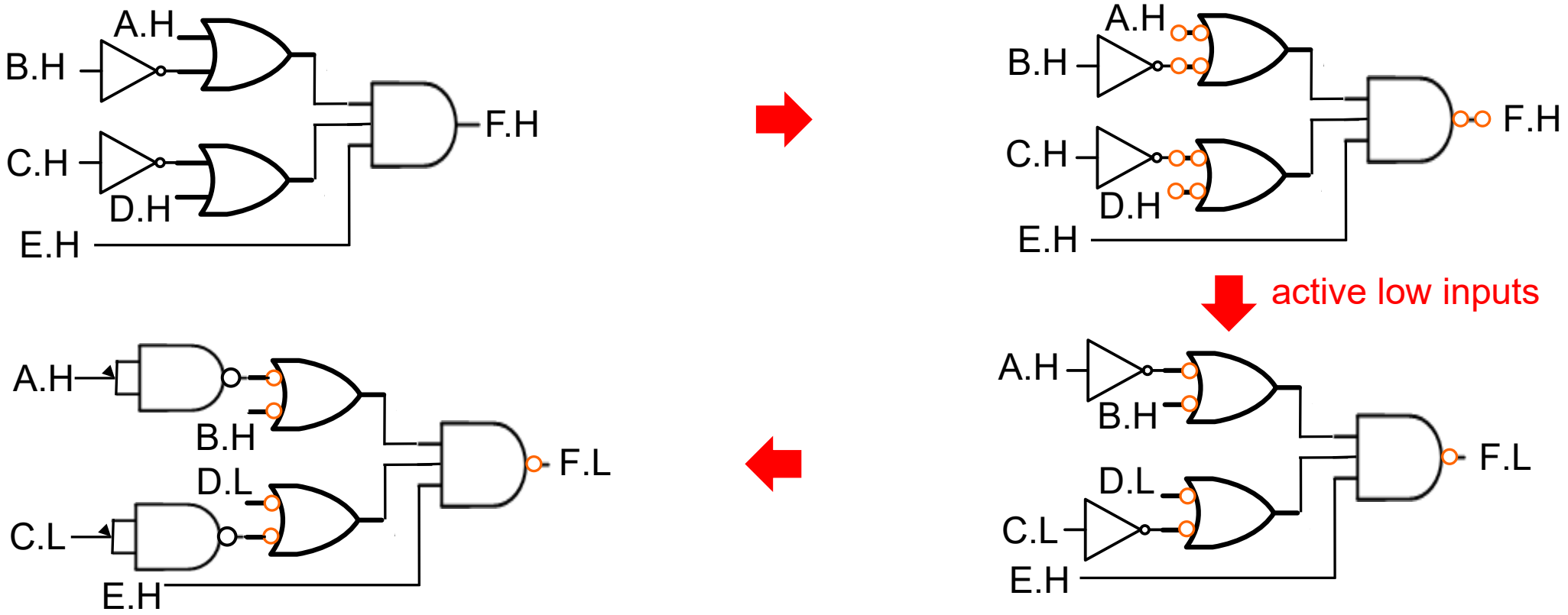
$$F = A\overline{B} + \overline{C}D$$



NAND only Implementation with Mixed Logic

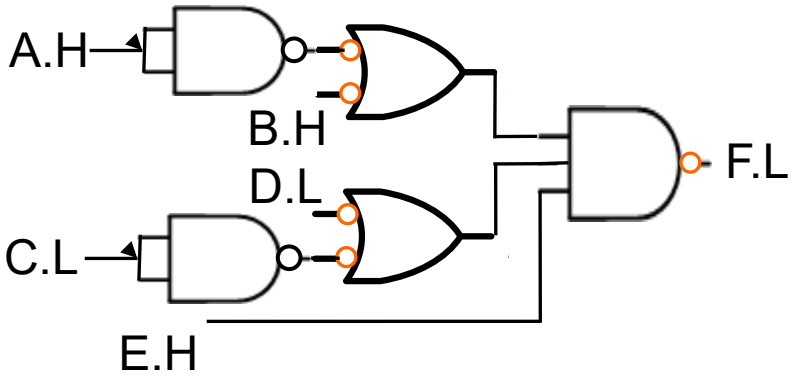
- Positive and negative logic are mixed (some signals are active-low)
  - Just design in positive logic and complement active-low signals
  - NAND-only example

$$F = (A + \overline{B}) \cdot (\overline{C} + D) \cdot E$$ (where C, D and F are active low)



NAND only Implementation with Mixed Logic

- Practice with another example
  - NAND- and NOR-only, expressing F in positive logic



Summary

- Karnaugh maps
- Boolean function simplification using K-map
  - SOP simplification
  - POS simplification
  - Don’t-care condition
  - Minimal SOP (MSOP) and POS (MPOS)
- Gate-level implementation
  - Various constraints (fan-in, library composition)
  - NAND-only, NOR-only
  - Include active-low signals